

Security Audit

Rizenet Token ("RIZE") (RWAXDeAI)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	11
Security Rating	13
Intended Smart Contract Behaviours	14
Code Quality	16
Audit Resources	16
Dependencies	16
Severity Definitions	17
Status Definitions	18
Audit Findings	19
Deployment Best Practices & Mainnet Review	25
Centralisation	28
Conclusion	29
Our Methodology	30
Disclaimers	32
About Hashlock	33

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The T-RIZE team partnered with Hashlock to conduct a security audit of their Rizenet Governance Token (“RIZE”) smart contracts. Hashlock manually and proactively reviewed the code to ensure the project’s team and community that the deployed contracts were secure.

Project Context

Rizenet is a public-permissioned Layer 1 blockchain developed by T-RIZE Group on Avalanche infrastructure. It is designed to support decentralized machine learning workflows.

The network is powered by the \$RIZE token, the core utility and governance asset of the ecosystem.

- Asset issuers may acquire, where legally permitted, and bond \$RIZE to mint and manage tokenized assets. The amount required varies depending on the distribution network and regulatory structure.
- Investors can use \$RIZE to access early-stage opportunities in select tokenized real-world assets and benefit from exclusive analytics and investment insights.
- \$RIZE is also required to:
 1. Cover gas fees and smart contract execution costs
 2. Participate in federated learning, as a contributor or consumer
 3. Operate validator nodes (subject to whitelisting)
 4. Vote on treasury allocations, ecosystem grants, and community incentives through a maturity-weighted bonding mechanism

Rizenet maintains a permissioned validator set and restricts smart contract deployment to authorized entities, supporting controlled participation. It is EVM-compatible, enabling Ethereum-standard tooling and smart contracts that enforce compliance conditions—such as identity-based transfer rules and jurisdictional constraints.

The network supports federated learning coordination through onchain logic. While model training and data remain offchain, smart contracts manage participant authorization, contribution tracking, and reward distribution, enabling privacy-preserving collaboration across institutional participants.

Together, Rizenet and the \$RIZE token provide the infrastructure, governance mechanism, and incentive structure required to support institutional-scale tokenization and decentralized machine learning.

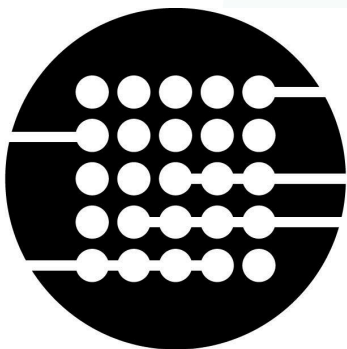
Project Name: Rizenet Token (RIZE)

GitHub Commit Hash: ee812128b198e25212c5e549fe253f9d17e957ff

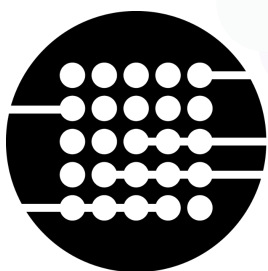
Compiler Version: ^0.8.24

Website: <http://t-rize.io>

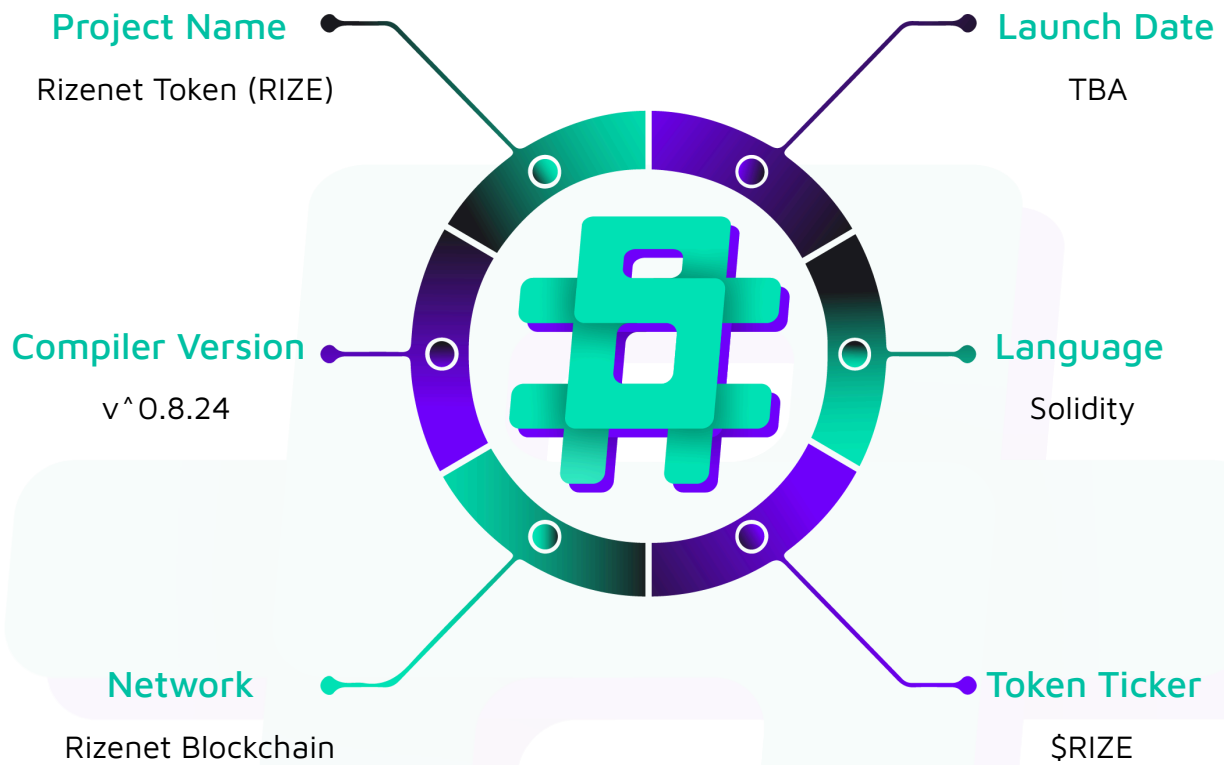
Logo:



**RIZE
TOKEN**



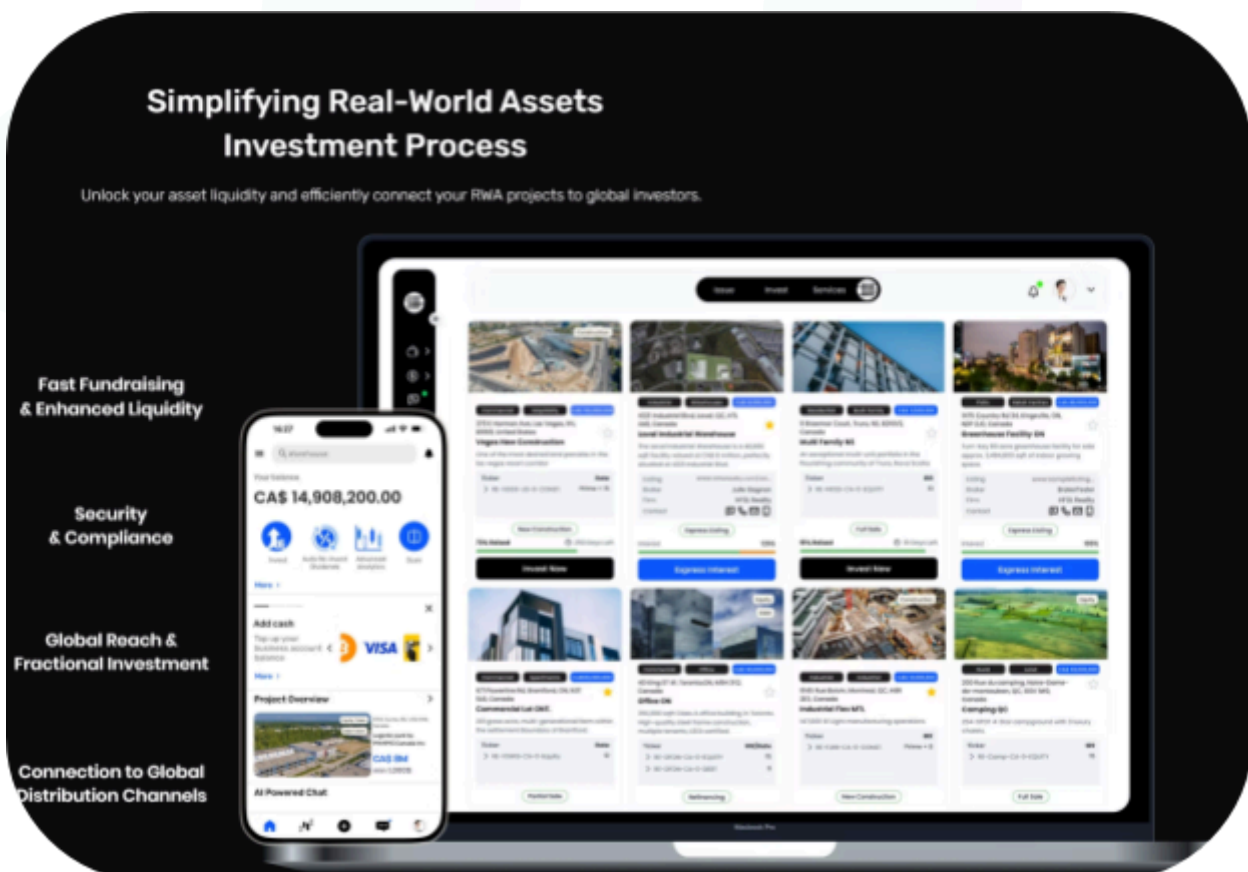
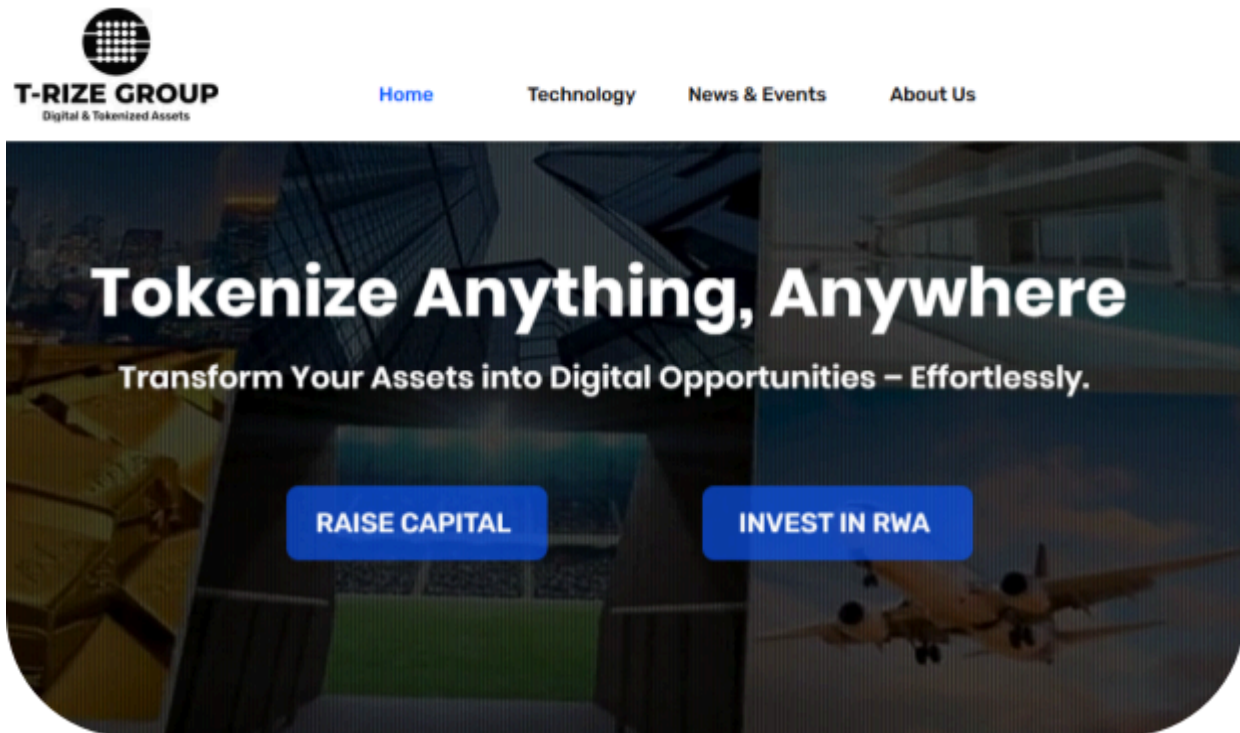
T-RIZE

Visualised Context:

Project Visuals:

T-RIZE Tokenization Platform:

T-RIZE Tokenization Platform Website (<https://www.t-rize.io/>)

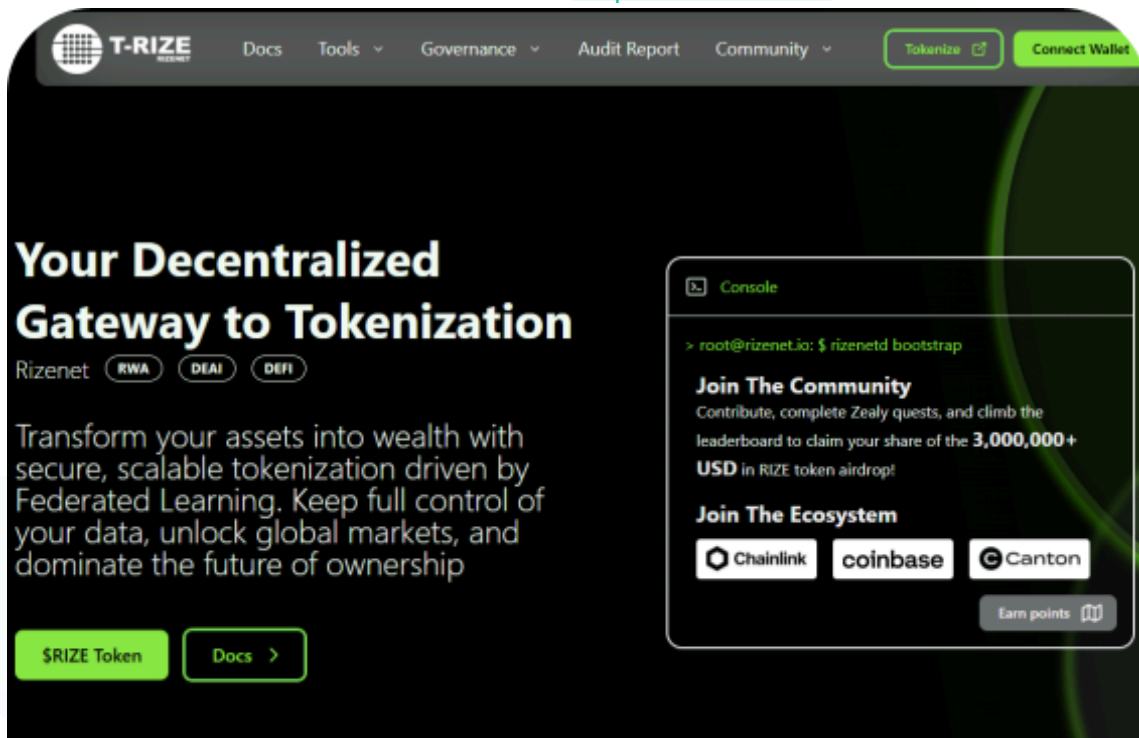


#hashlock.

Hashlock Pty Ltd

Rizenet Portal:

Rizenet Website (<https://rizenet.io/>)

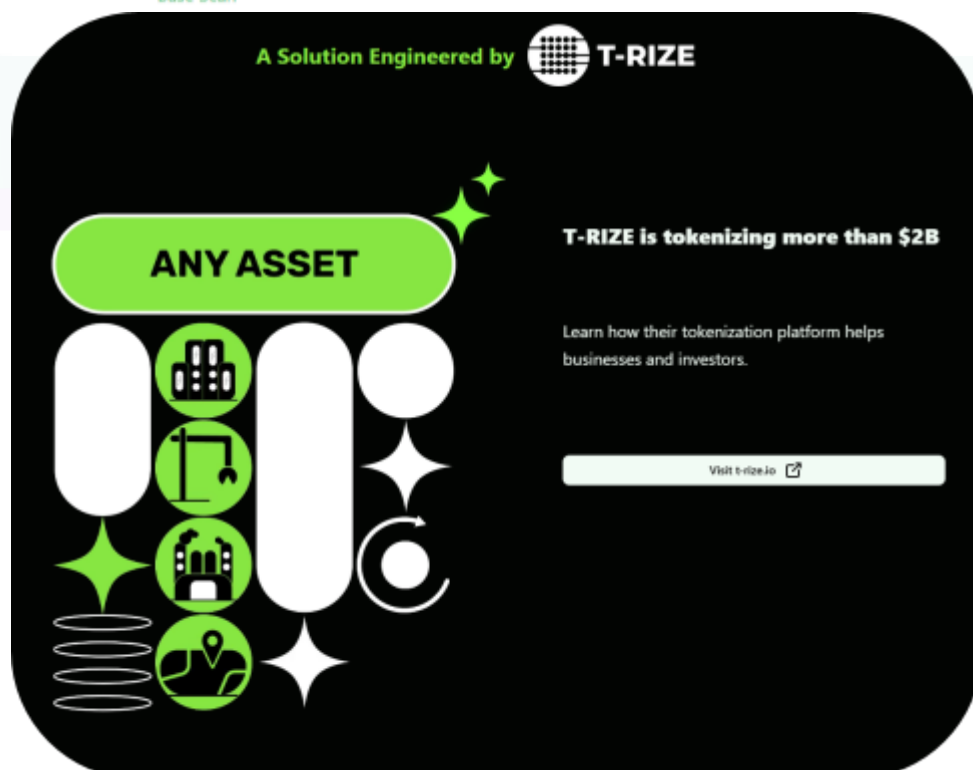


\$23M \$300M \$2B+

Total Value Locked
DeFiLlama
Base Scan

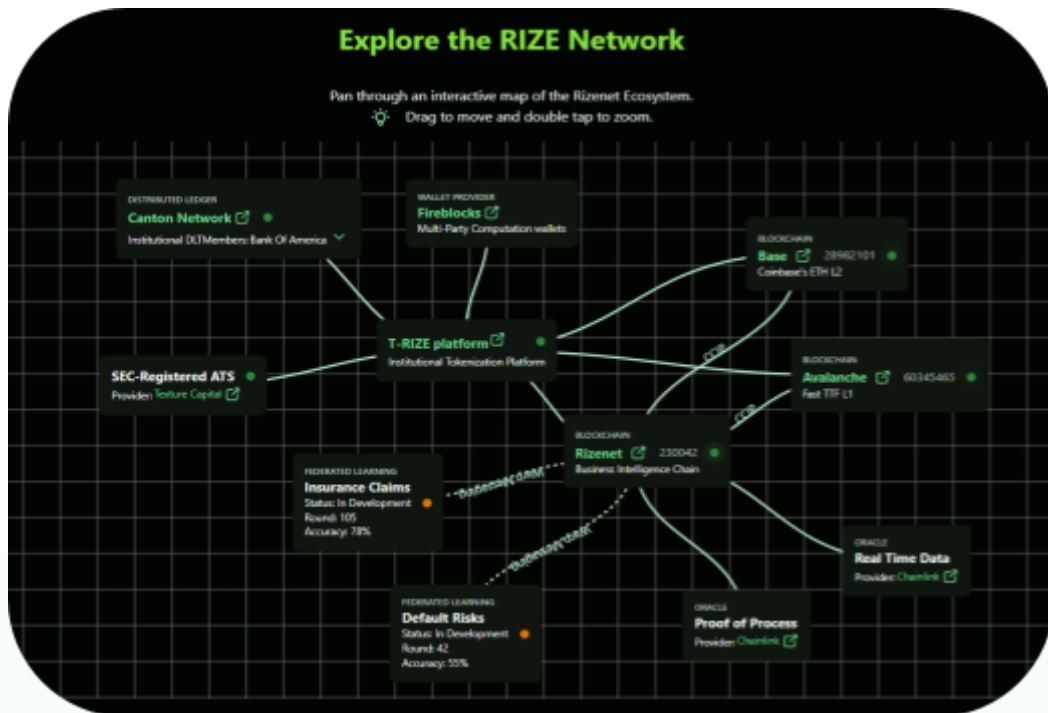
Upcoming Total Value Locked
[Learn More](#)

Signed MOUs



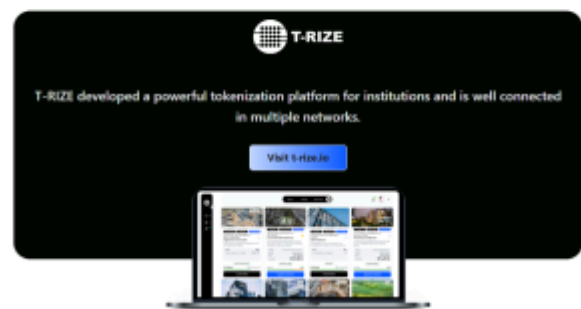
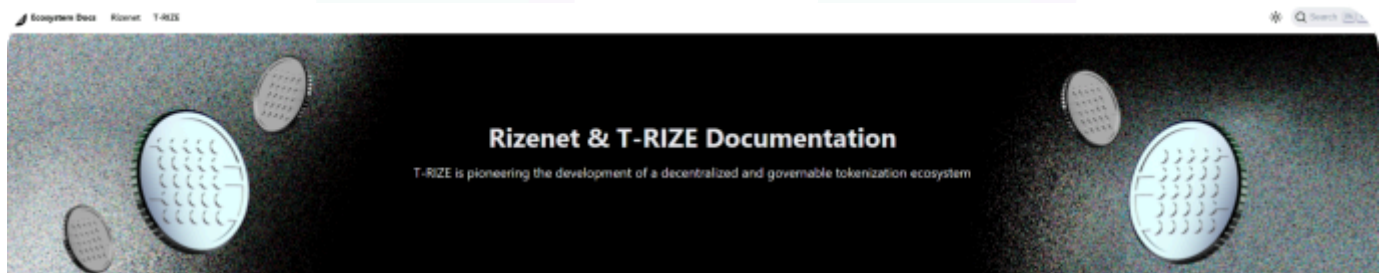
#hashlock.

Hashlock Pty Ltd



Rizenet & T-RIZE Documentation:

Ecosystem Docs (<https://docs.t-rize.io/>)



Overview

- Getting Started
 - Connecting to Rizenet
 - Smart Contract Deployer Allowlist
 - Token Faucet
- Collective Intelligence
 - Overview
- Rizenet Blockchain
 - Public-Permissioned Architecture
 - Applications
 - Design Requirements
 - Validators
 - Scaling Security with TVL
 - Validator Requirements
 - Node Requirements
 - Node Deployment
 - Node Updating
 - Node Status Monitoring
 - Common Node Operations
- RIZE Token
 - Introduction
 - Token Utility
 - Staking
 - Audits
 - Governance
 - Governance Mechanisms
 - Participating in Governance
 - Maturity-Adjusted Voting Weights
 - Governance NFT
 - Token Dynamics and Treasury Management
 - MS committee
 - Developer Resources

Overview

Overview

About Rizenet

Rizenet is a public-permissioned Layer 1 blockchain developed by T-RIZE Group on *Avalanche infrastructure*. It is designed for decentralized machine learning workflows.

The network operates with a permissioned validator set and restricts smart contract deployment to authorized entities, ensuring operational control and regulatory compliance. It is EVM-compatible, allowing the use of Ethereum-standard tooling and enabling asset-level compliance logic—such as identity-based transfer restrictions or jurisdictional enforcement—via smart contracts.

Rizenet supports federated learning coordination through onchain logic. While model training and data handling occur off-chain, smart contracts manage participant authorization, contribution tracking, and reward distribution. This enables privacy-preserving collaboration across institutional actors.

The native token, \$RIZE, serves as the core utility and governance asset. It is used to:

- Pay gas fees and smart contract execution costs
- Access tokenization and lifecycle management services on T-RIZE platform
- Unlock curated RWA deal flow and institutional infrastructure
- Participate in federated learning (as contributor or consumer)
- Operate validator nodes (subject to whitelisting)
- Govern protocol upgrades, treasury usage, and network policy via token bonding

\$RIZE provides the economic layer for infrastructure access, coordination, and governance across the Rizenet ecosystem.

Since March 2024, the Rizenet testnet has been operational and under active development. Engineers are testing its capabilities. Validators such as the ETS university are providing operational feedback. The research partners from the T-RIZE Labs are working on pushing its abilities. The mainnet will be launched once all testing is completed.

Why build with us?

T-RIZE stands at the forefront of innovation with its fully integrated, end-to-end technology stack, making it the ultimate platform for asset issuers to seamlessly launch real-world asset (RWA) tokenization projects and for investors to onboard capital effortlessly. With robust integrations spanning both asset management, compliance and payments, we simplify every step of the tokenization process for issuers and investors.

Our no-code, end-to-end solution empowers RWA projects to focus entirely on their product by abstracting blockchain related complexities. The investors benefit from a streamlined, secure platform that allows them to allocate capital with confidence. This comprehensive approach fosters a vibrant ecosystem where both issuers and investors can thrive, driving innovation, removing barriers, and enabling seamless collaboration within the Rizenet network.

About Rizenet
Why build with us?
T-RIZE Team

Introduction

- Tokenization
 - About RWA
 - Tokenization Use Cases
- T-RIZE Platform
 - Tokenization Process
 - Token Types
 - Compliance
 - ERC-3643 Standard
 - Distribution
 - Platform Wallets
 - Blockchain Support
 - RIZE Token Integration
 - Platform Integrations
 - Verifications Mechanisms
 - U.S. Market Access
- T-RIZE Labs
 - About T-RIZE Labs
 - T-RIZE Labs Ongoing Research
 - Decentralized Machine Learning
- Partnerships
 - Collaborations & Partnerships
 - Community
- Team

Introduction

Introduction

About T-RIZE

T-RIZE Group is at the forefront of revolutionizing the accessibility of traditionally illiquid assets, facilitating the seamless trading of Real-World Assets, ranging from real estate ventures to verified carbon credits. Built on four core pillars, T-RIZE orchestrates a harmonious system that ensures the tokenization and management of high-value assets are secure, scalable, and efficient.

The T-RIZE Tokenization Platform unlocks liquidity, reaches global investors, and accelerates fundraising by bridging decentralized technologies with traditional finance, facilitating fractional ownership of limitless asset classes through digital tokenization.

The Rizenet Blockchain, powered by the Rizenet (RIZE) governance tokens, is the core of this ecosystem, designed to optimize the value of tokenized assets by facilitating their distribution and management. Rizenet is a public-permissioned *Avalanche L1* optimized for asset tokenization. It uses the Ethereum Virtual Machine (EVM) as the smart contract execution engine and leverages the *Avalanche Warp Messaging Protocol* to provide secure and reliable business intelligence for decision making regarding tokenized assets.

T-RIZE Labs drives ongoing Research and Development in Blockchain and Decentralized Machine Learning through its partnership with the engineering university École de Technologie Supérieure (ÉTS), establishing one of the world's first industrial research chairs focused 100% on Tokenization and Decentralized Machine Learning ensuring the platform and its ecosystem continuous innovation and growth.

Through its robust **Institutional Distribution Network**, T-RIZE Group is uniquely positioned to democratize access to premium asset ownership. It stands committed to reshaping the investment landscape by harnessing the power of tokenization, supported by its deep institutional connections and compliant asset distribution.

The T-RIZE Solution

T-RIZE stands in contrast to conventional funding avenues by offering streamlined access to a diverse array of both traditional and digital assets. This approach not only amplifies operational efficiency but also curtails costs dramatically by forging a direct connection between issuers and investors. Through the integration of cutting-edge technologies, T-RIZE simplifies the investment procedure and unveils a world of new opportunities for global investors and asset owners alike.

- 50% Faster Financing:** By leveraging advanced technology and streamlined processes, T-RIZE accelerates access to capital, enabling asset owners to raise funds 50% faster compared to traditional methods.
- 40%-90% Cost Savings:** By reducing intermediaries and automating complex financial transactions, T-RIZE significantly cuts costs for both investors and asset owners, making the entire process more economical and efficient.
- Global Reach:** With an extensive network of institutional channels and global investors, T-RIZE provides unmatched market access, enabling asset owners to tap into a worldwide investor base.

By integrating cutting-edge technologies and incorporating ongoing research, T-RIZE paves the way for boundless possibilities in the future of global trading. Through the tokenization of real-world assets (RWA) on Rizenet, T-RIZE effectively mitigates the hurdles faced in conventional real-world asset markets.

About T-RIZE
The T-RIZE Solution
The T-RIZE Team

Audit scope

We at Hashlock audited the solidity code within the Rizenet Token (RIZE) project, the scope of work included a comprehensive review of the smart contracts listed below. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

These tests primarily include the manual line-by-line analysis step and on-chain provable vulnerability testing step with Hardhat framework.

We went through an in-depth manual review of the codebase, starting with a review of common vulnerabilities such as reentrancy, integer overflow/underflow, unchecked external calls, and lack of access control.

Description	T-RIZE Smart Contracts
Platform	RIZENET / Solidity
Audit Date	March, 2025
Contract 1	AccessList.sol
Contract 1 MD5 Hash	29bc6cf5b40b029cd58e5e9f3ec874d0
Contract 2	AccessRestricted.sol
Contract 2 MD5 Hash	b5e98e4294c4374c41e66677e2eb5f1c
Contract 3	BondVesting.sol
Contract 3 MD5 Hash	f557572d6c7d6f99eac4263c3b7e46ba
Contract 4	EIP3009.sol
Contract 4 MD5 Hash	db9ffebb815792068bfa305daae69f44
Contract 5	GovernanceBonding.sol
Contract 5 MD5 Hash	82dc53cc1c782012ac6e29627314f801
Contract 6	MaturityPools.sol
Contract 6 MD5 Hash	78ad45534edb8415e1fd2f4b47df75b8
Contract 7	NFTBond.sol

Contract 7 MD5 Hash	a05d9beeb31cf059c132b3e2d815476d6
Contract 8	RizeToken.sol
Contract 8 MD5 Hash	b2696396141782989d040c3bd52c1c4c
Contract 9	WeightedBonds.sol
Contract 9 MD5 Hash	b4017c72e2928329a93bcb71108ce216
Contract 10	NativeRizeToken.sol
Contract 10 MD5 Hash	ac0676112b3ab5fbc1efbd21dcb8b1
Contract 11	MerkleAirdrop.sol
Contract 11 MD5 Hash	5b2a2d19b2782b6de44ea83f4ad77546
Contract 12	MerkleDistributor.sol
Contract 12 MD5 Hash	fce8bdfd9ca5f92fc01e4fe12b34c30a
Contract 13	UnallocatedVesting.sol
Contract 13 MD5 Hash	41457fc0a3f8c4e91beff474d479bb93
Contract 14	VestingERC20.sol
Contract 14 MD5 Hash	5339d7f9931db28189865829412f14be
GitHub Commit Hash	575d1ae3d5ffb73346e8638d305ef859e082fe8c
Fix Review GitHub Commit Hash	ee812128b198e25212c5e549fe253f9d17e957ff

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

- 3 Low-severity vulnerabilities
- 6 QAs
- 3 Gas Optimizations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
AccessList.sol Contains addresses of system contracts and roles.	Contract achieves this functionality.
AccessRestricted.sol Checks if the caller is a committee and a bridge.	Contract achieves this functionality.
BondVesting.sol Tracks vesting of tokens using a linear unlock: - Tokens are fully vested until the cliff ends - Tokens are unlocked linearly from the cliff until the end of the vesting period - Tokens are fully unlocked past the vesting	Contract achieves this functionality.
EIP3009.sol Internal implementation for gas-abstracted transfers.	Contract achieves this functionality.
GovernanceBonding.sol Allows committee to: - Define the time required between breaking the bond and calling the release function - Add a pool - Update the parameters of a pool - Revoke vested tokens of a bond - Updates the vesting schedule - Add/Remove migrator	Contract achieves this functionality.
MaturityPools.sol Contains information about the different pools and their weights.	Contract achieves this functionality.

NFTBond.sol NFT to represent the governance bonds.	Contract achieves this functionality.
RizeToken.sol WETH9 equivalent on Rizenet, but a standard ERC20 on other chains.	Contract achieves this functionality.
WeightedBonds.sol Custody tokens and stores bond related information.	Contract achieves this functionality.
MerkleAirdrop.sol Allows the MS Committee to upload a merkle tree root. Users can claim their allocations as vested bonds.	Contract achieves this functionality.
UnallocatedVesting.sol Enables the owner to transfer vested tokens. The owner can extend the vesting terms.	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts for the Rizenet Token (RIZE) project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Rizenet Token (RIZE) smart contract code in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help understand the overall architecture of the protocol.

Dependencies

Per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry-standard open-source projects.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Low

[L-01] RizeToken#_withdraw - Deprecated Ether transfer function

Description

The `_withdraw` function uses the `.transfer` function to transfer native coins.

The Istanbul update made some changes to the EVM, which made the `.transfer` function deprecated for the ETH transfer.

Recommendation

Use the `.call` function for the ETH transfer and check the return value.

Status

Resolved

[L-02] RizeToken#setFeesBps - Lack of emitting an event

Description

The `setFeesBps` function is missing an event when a key storage is updated.

Recommendation

Add a relevant event in the function.

Status

Resolved

[L-03] UnallocatedVesting - Use Ownable2Step versions rather than Ownable versions

Description

Ownable2StepUpgradeable and Ownable2Step prevent the contract ownership from mistakenly being transferred to an address that cannot handle it (e.g. due to a typo in the address), by requiring that the recipient of the owner permissions actively accept via a contract call of its own.

Recommendation

Consider using Ownable2StepUpgradeable and Ownable2Step from OpenZeppelin Contracts to enhance the security of your contract ownership management. This contract prevents the accidental transfer of ownership to an address that cannot handle it, such as due to a typo, by requiring the recipient of owner permissions to actively accept ownership via a contract call.

Status

Resolved

QA

[Q-01] AccessList#setGovernanceBonding, setBonds, setToken, setTreasury - Missing zero address checks

Description

The setGovernanceBonding, setBonds, setToken, and setTreasury functions are missing zero address checks for the input parameters values.

Recommendation

Add checks if the input parameters values are zero.

Status

Resolved

[Q-02] WeightedBonds - Unused struct

Description

The `BondWithId` struct is not used at all in the contract.

Recommendation

Remove the struct.

Status

Resolved

[Q-03] WeightedBonds - Readability improving

Description

The `_releaseWarmup` is set to `7 * 24 * 60 * 60` but it reduces the readability.

Recommendation

Replace `7 * 24 * 60 * 60` with `7 days`.

Status

Resolved

[Q-04] MaturityPools - Unnecessary assignment

Description

The `_poolId` is set to 0 again, even if its default value is 0 as standard.

Recommendation

Avoid assigning to 0 when declaring.

Status

Resolved

[Q-05] NativeRizeToken - Unused variable declaration

Description

The NativeRizeToken contract contains `_feesBps` variable that has been declared but is not utilized within the contract's logic.

Recommendation

Remove unused variables.

Status

Resolved

[Q-06] UnallocatedVesting - Redundant private variable declaration

Description

The UnallocatedVesting contract declares a private variable (`_accessList`) with the same name as a private variable in the parent contract (VestingERC20). Since private variables in Solidity are not accessible outside the contract in which they are declared, the child contract's variable is entirely independent of the parent's variable. This redundancy can lead to confusion for developers and increases the risk of errors during maintenance or upgrades. Additionally, the variable in question is assigned a value but is never used, which further indicates unnecessary resource allocation.

Recommendation

Remove the `_accessList` private variable declaration in the UnallocatedVesting contract.

Status

Resolved

Gas

[G-01] AccessRestricted, WeightedBonds - `_accessList` and `_govBond` variables could be made immutable

Description

The `_accessList` variable in the `AccessRestricted` contract is only set in the constructor.

Same for the `_govBond` variable in the `WeightedBonds` contract.

Recommendation

Make the variables immutable.

Status

Resolved

[G-02] WeightedBonds#release - `release.length` is read multiple times

Description

In the `release` function, `release.length` is read at every iteration from storage.

Recommendation

Cache `release.length` into a memory variable.

Status

Resolved

[G-03] VestingERC20 - `_accessList` and `_token` variables could be made immutable

Description

The `_accessList` and `_token` variables in the `VestingERC20` contract are only set in the constructor.

Recommendation

Make the variables immutable.

Status

Resolved



Deployment Best Practices & Mainnet Review

Following the remediation of all audit findings, Hashlock has conducted a deployment review of the Rizenet (\$RIZE) smart contracts to assess whether the current mainnet deployment aligns with the audited codebase and best practices for production readiness.

Deployment Review Summary:

Based on the review of the live deployment:

1. Contracts are deployed with the correct parameters:
All constructor arguments and initial configuration settings match the expected values from the audited codebase.
2. Tokens have been correctly allocated:
The \$RIZE tokens have been successfully distributed to the correct contracts and addresses as per the deployment plan.

These findings confirm that the deployment is technically aligned with the audited version, and no discrepancies were observed in the parameters or token distribution.

Recommendations and Best Practices:

1. Multi-Signature Wallets

We recommend, as best practices for deployment, the ownership and treasury management to multi-signature wallets (e.g., Gnosis Safe) with an appropriate number of signers. This is essential to:

- Mitigate single-point-of-failure risks
- Improve governance transparency
- Protect against unauthorized administrative actions

To this day, the T-RIZE team has the administrative control over the deployed contracts currently managed through Fireblocks' MPC infrastructure, which enforces separation of roles (initiator, approver, signer) and is supported by Station70 for institutional-grade backup and recovery.

Treasury assets are stored offline using Ledger hardware wallets, secured in physically protected vaults that are both waterproof and fireproof. These wallets are managed by institutional entities, distributed across multiple secure locations, and maintained in compliance with international security protocols. While the setup does not rely on on-chain multi-signature contracts, it achieves equivalent security through enforced multi-actor control, effectively eliminating single points of failure.

The T-RIZE team is also committed to transitioning to a Safe multi-signature wallet with diverse signers before the end of the 12-month governance cliff following TGE. This transition will align with broader decentralization goals and include the onboarding of institutional ecosystem participants such as ETS University, contributors, and validators through community governance.

2. Post-Deployment Bug Bounty Program:

While the Rizenet (\$RIZE) contracts have undergone a thorough audit and all known vulnerabilities have been remediated, no audit can guarantee the absolute absence of bugs or unforeseen edge cases. Smart contract security is a continuous process, particularly once contracts are deployed and interacting with real assets and users in a live environment.

We strongly encourage the T-RIZE team to launch a public bug bounty program as a follow-up to this audit. This offers several key benefits:

- **Crowdsourced Security:** Taps into the broader security research community to uncover vulnerabilities that may not surface during static audits.
- **Real-World Testing Context:** Encourages testing of live contracts under real-world conditions and adversarial scenarios.

- **Transparency and Trust:** Signals to the ecosystem and stakeholders that the project takes long-term security seriously.
- **Incentivized Disclosure:** Reduces the likelihood of zero-day exploits by rewarding responsible disclosure of bugs.

Hashlock manages submissions and rewards to ensure robust security practices for enrolled bug bounty programs. We recommend clearly defining scopes, severity tiers, and payout structures, along with maintaining transparent communication with the community regarding reported and resolved issues.

By complementing the audit with a bug bounty program, the T-RIZE team can significantly reinforce the security posture of the Rizenet ecosystem and build a trusted foundation for long-term growth.

Centralisation

The Rizenet Token (RIZE) project values security and utility over decentralisation.

The Rizenet Token (RIZE) and governance contracts relies on an address labelled as “committee” with the power to:

- Configure governance pools with arbitrary parameters
- Break soul binding spell on the governance NFT
- Update the time required to unlock bonded tokens from 0 to 31 days
- Configure bridge contracts with the ability to mint RIZE ERC20 on non-native chains

These centralization points can be mitigated by using a multi-signature wallet with a timelock contract, and T-RIZE will transition to a Safe multi-signature wallet.

The committee cannot blacklist or transfer tokens on behalf of others.

The vesting initiator can clawback vested tokens.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Rizenet Token (RIZE) project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.